CO2-AT2

NAME:POLI REDDY
REGNO:192325056

COURSE NAME:DEEP LEARNING

COUSE CODE:MLA0401

1. Unsupervised Training of Neural Networks, Auto Encoders

A hospital wants to identify abnormal ECG patterns from thousands of unlabeled

records. Which deep learning technique from the syllabus would you recommend?

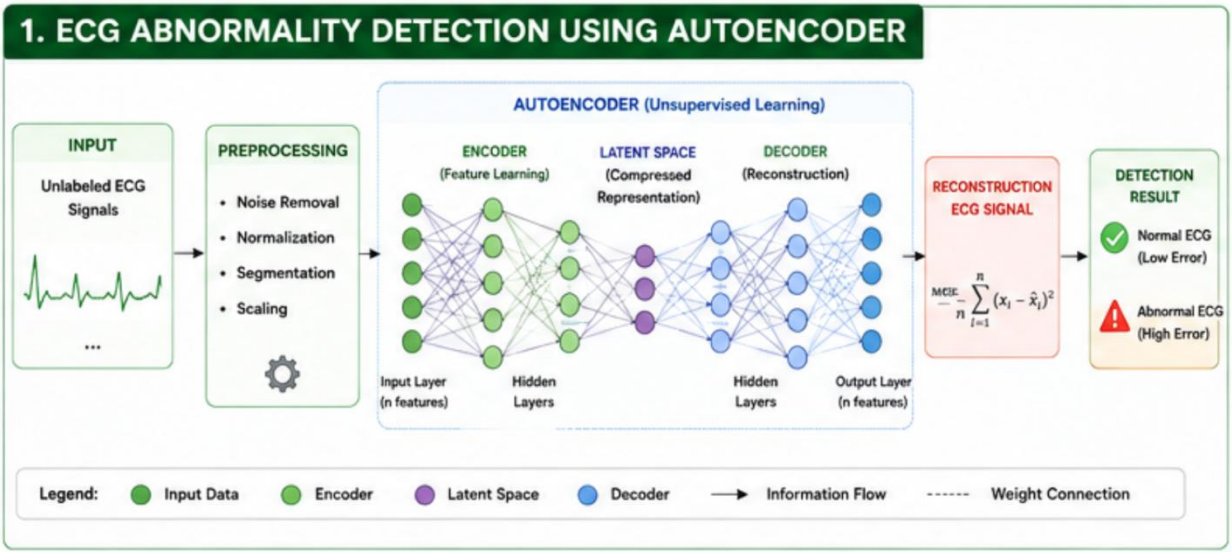
Justify your choice and explain the workflow.

Explanation:

Hospitals have thousands of **unlabeled ECG signals**.

An **Autoencoder** learns the normal ECG patterns by compressing and reconstructing the signals.
After reconstruction, the error between the original and reconstructed ECG is calculated.
Low reconstruction error indicates a normal ECG.
High reconstruction error indicates an abnormal ECG.

Work flow diagram:



DATA SET:

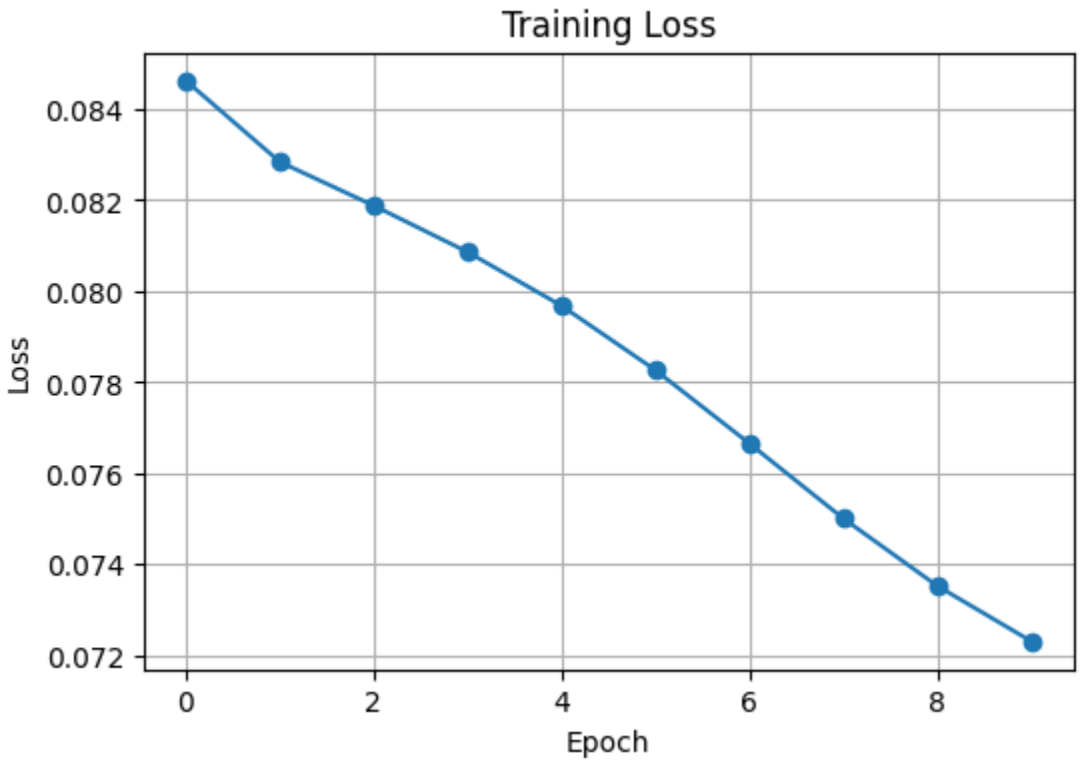
```
ECG Signal for file_index = 0, [Function_Name:(p_signal)] :
[[-0.06 ]
 [-0.065]
 [-0.06 ]
 ...
 [ 0.   ]
 [ 0.   ]
 [ 0.   ]]

Annotation Symbols for file_index = 0 [Function_Name:(atr_sym)] :
['N', 'N', 'N', 'N', 'N', 'N', 'N', 'N', 'N', 'N', 'N', 'N', 'N', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A

Annotation Sample Indices for file_index = 0 [Function_Name:(atr_sample)] :
[      0    6000   12000   18000   24000   30000   36000   42000   48000
   54000   60000   66000   72000   78000   84000   90000   96000  102000
  108000  114000  120000  126000  132000  138000  144000  150000  156000
  162000  168000  174000  180000  186000  192000  198000  204000  210000
  216000  222000  228000  234000  240000  246000  252000  258000  264000
  270000  276000  282000  288000  294000  300000  306000  312000  318000
  324000  330000  336000  342000  348000  354000  360000  366000  372000
  378000  384000  390000  396000  402000  408000  414000  420000  426000
  432000  438000  444000  450000  456000  462000  468000  474000  480000
  486000  492000  498000  504000  510000  516000  522000  528000  534000
  540000  546000  552000  558000  564000  570000  576000  582000  588000
  594000  600000  606000  612000  618000  624000  630000  636000  642000
...
 2754000 2760000 2766000 2772000 2778000 2784000 2790000 2796000 2802000
 2808000 2814000 2820000 2826000 2832000 2838000 2844000 2850000 2856000
 2862000 2868000 2874000 2880000 2886000 2892000 2898000 2904000 2910000
 2916000 2922000 2928000]
```

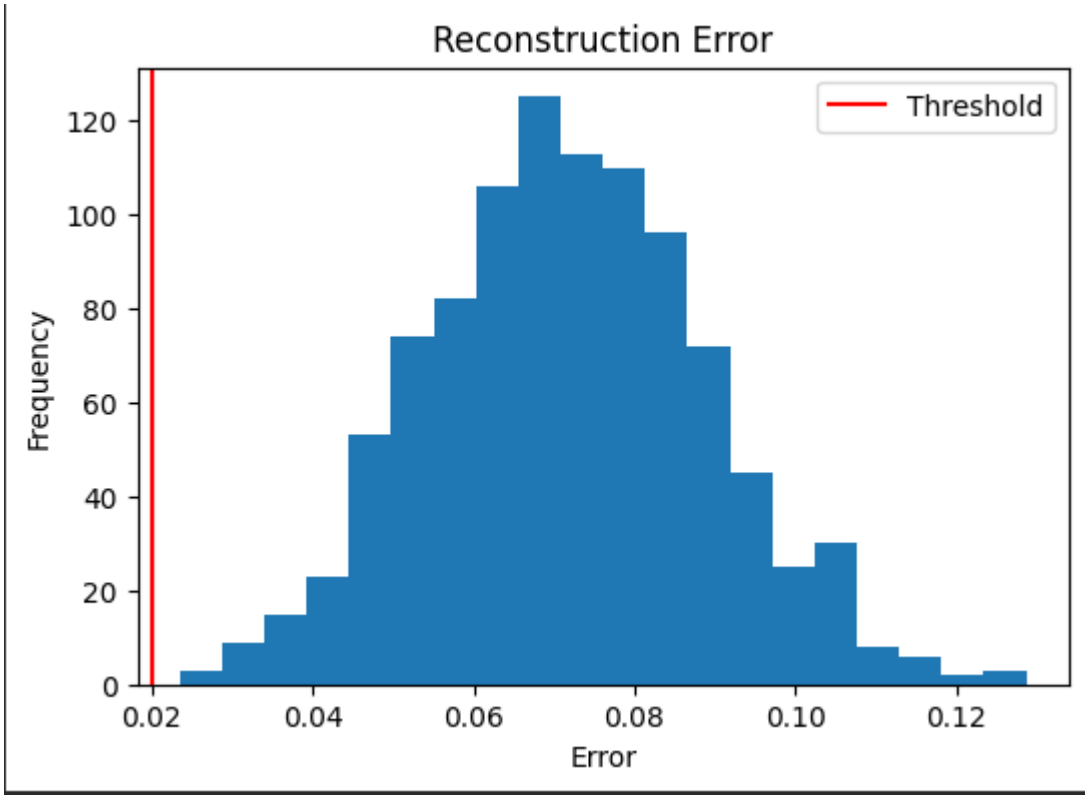
python code:

```python
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Generate sample ECG data
X = np.random.rand(1000, 20)

# Autoencoder Model
model = Sequential([
    Dense(10, activation='relu', input_shape=(20,)),
    Dense(5, activation='relu'),
    Dense(10, activation='relu'),
    Dense(20, activation='sigmoid')
])

model.compile(optimizer='adam', loss='mse')

# Train
history = model.fit(X, X, epochs=10, batch_size=32, verbose=1)

# Plot Training Loss
plt.figure(figsize=(6,4))
plt.plot(history.history['loss'], marker='o')
plt.title("Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid(True)
```

```
plt.show()

# Reconstruction
pred = model.predict(X)

# Reconstruction Error
error = np.mean((X - pred)**2, axis=1)

# Plot Histogram
plt.figure(figsize=(6,4))
plt.hist(error, bins=20)
plt.axvline(0.02, color='red', label='Threshold')
plt.title("Reconstruction Error")
plt.xlabel("Error")
plt.ylabel("Frequency")
plt.legend()
plt.show()
```

OUTPUT:





2. Auto Encoders, Representation Learning

A retail company wants to reduce the dimensionality of customer purchase data

before performing customer segmentation. Explain how an autoencoder can be used

to solve this problem.

EXPLANATION:

Customer purchase data contains many features, making analysis difficult.
An **Autoencoder** reduces the data into a smaller set of important features.
These compressed features preserve customer buying behavior.

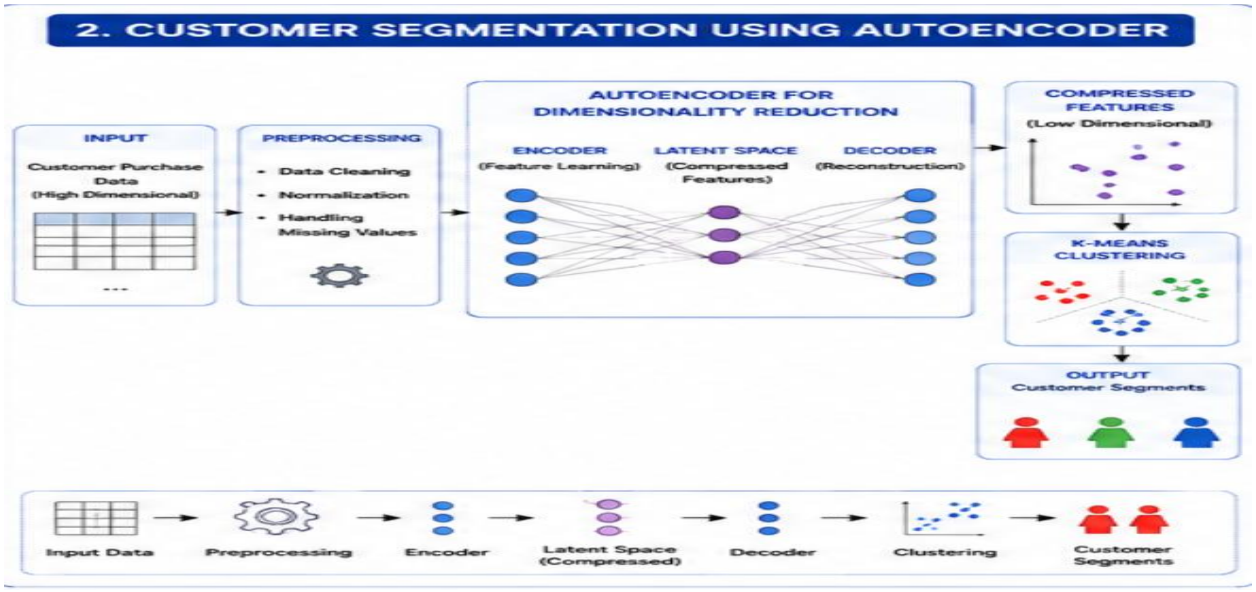
The reduced data is then given to **K-Means clustering**.
Customers are grouped into different segments for targeted marketing.

DATASET:

Dataset Name: `customer_purchase.csv`

CustomerID	Age	Income	SpendingScore	Purchases	OnlineOrders
101	25	30000	80	25	18
102	32	45000	60	18	10
103	45	70000	35	12	8
104	28	35000	75	20	16
105	50	85000	25	8	4
106	38	55000	55	15	12
107	29	40000	78	22	19
108	41	65000	40	13	9
109	36	52000	58	17	13
110	31	48000	68	19	15

WORK FLOW:



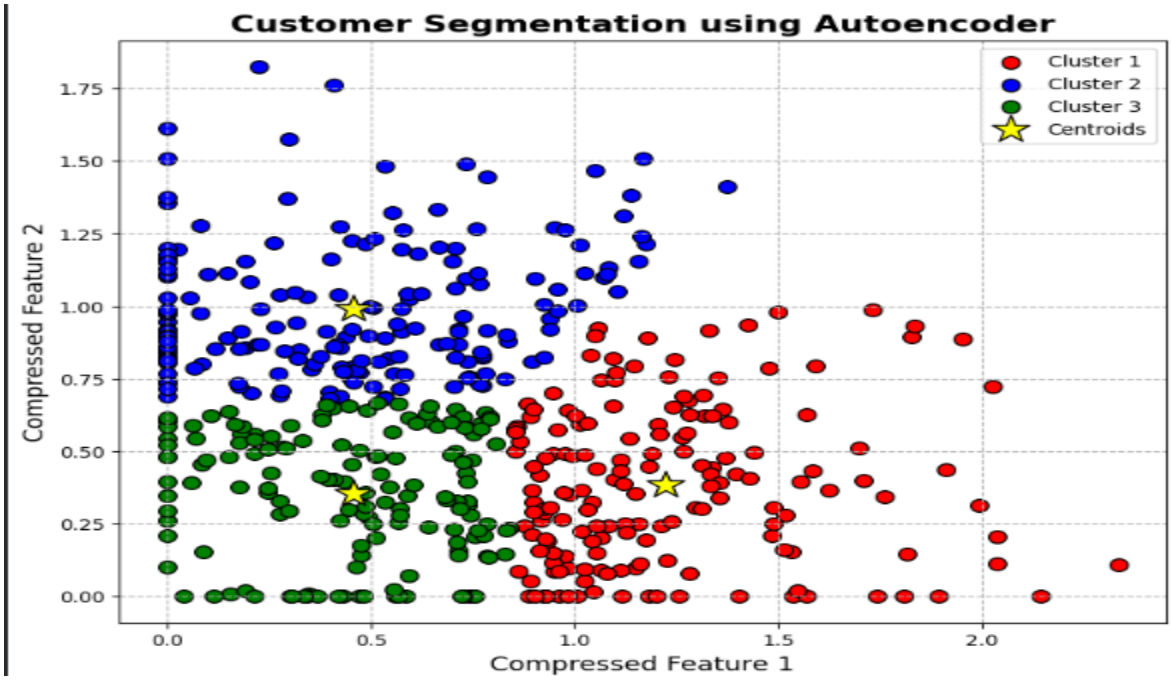
PYTHON CODE:

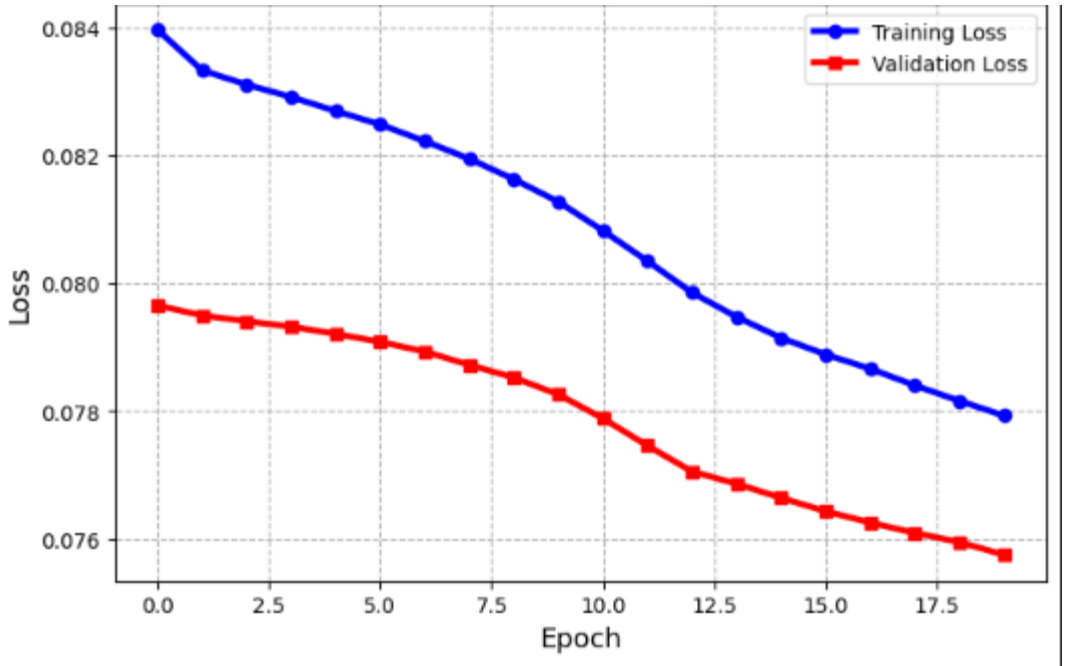
```
# Install libraries (Run only once in Google Colab)
# !pip install tensorflow scikit-learn matplotlib

import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model
from sklearn.cluster import KMeans


np.random.seed(42)

# 500 Customers and 20 Purchase Features
X = np.random.rand(500,20)

print("Customer Dataset Shape :",X.shape)


input_layer = Input(shape=(20,))

# Encoder
encoded = Dense(12,activation='relu')(input_layer)
encoded = Dense(2,activation='relu')(encoded)

# Decoder
decoded = Dense(12,activation='relu')(encoded)
decoded = Dense(20,activation='sigmoid')(decoded)

# Models
autoencoder = Model(inputs=input_layer,outputs=decoded)
```

```
encoder = Model(inputs=input_layer,outputs=encoded)


autoencoder.compile(optimizer='adam',
                    loss='mse')


history = autoencoder.fit(
    X,
    X,
    epochs=20,
    batch_size=16,
    validation_split=0.2,
    verbose=1
)

compressed = encoder.predict(X)

print("\nCompressed Data Shape :",compressed.shape)


kmeans = KMeans(n_clusters=3,random_state=42)

clusters = kmeans.fit_predict(compressed)

centers = kmeans.cluster_centers_


plt.figure(figsize=(9,7))

colors=['red','blue','green']

for i in range(3):

    plt.scatter(
        compressed[clusters==i,0],
        compressed[clusters==i,1],
        s=70,
        color=colors[i],
        edgecolors='black',
        label='Cluster '+str(i+1)
    )

plt.scatter(
    centers[:,0],
    centers[:,1],
    s=350,
    color='yellow',
    marker='*',
    edgecolors='black',
    label='Centroids'
)

plt.title("Customer Segmentation using Autoencoder",
```

```
          fontsize=16,
          fontweight='bold')

plt.xlabel("Compressed Feature 1",fontsize=13)
plt.ylabel("Compressed Feature 2",fontsize=13)

plt.grid(True,linestyle='--')

plt.legend()

plt.show()


plt.figure(figsize=(8,5))

plt.plot(history.history['loss'],
         color='blue',
         linewidth=3,
         marker='o',
         label='Training Loss')

plt.plot(history.history['val_loss'],
         color='red',
         linewidth=3,
         marker='s',
         label='Validation Loss')

plt.title("Autoencoder Training Loss",
          fontsize=16,
          fontweight='bold')

plt.xlabel("Epoch",fontsize=13)

plt.ylabel("Loss",fontsize=13)

plt.grid(True,linestyle='--')

plt.legend()

plt.show()

print("\nCustomer Segmentation Results\n")

for i in range(3):

    print("Cluster",i+1,"contains",
          np.sum(clusters==i),
          "customers")

print("\nCustomer Segmentation Completed Successfully.")
```

OUTPUT:


Customer Segmentation using Autoencoder
Cluster 1
Cluster 2
Cluster 3
Centroids
Compressed Feature 2
Compressed Feature 1
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
0.0
0.5
1.0
1.5
2.0




```
Customer Segmentation Results

Cluster 1 contains 166 customers
Cluster 2 contains 182 customers
Cluster 3 contains 152 customers

Customer Segmentation Completed Successfully.
```

3. Width and Depth of Neural Networks, Activation Functions

A facial recognition system performs poorly when trained using a shallow neural

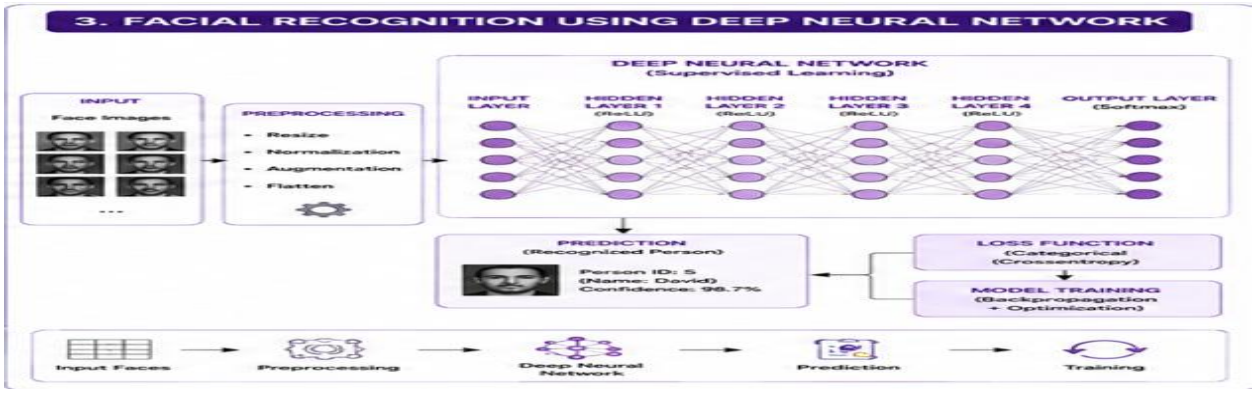
network. Recommend architectural changes involving network depth and activation

functions

EXPLANATION:

A shallow neural network cannot learn complex facial features effectively.
A **Deep Neural Network (DNN)** with multiple hidden layers extracts detailed facial features.
ReLU activation speeds up learning and avoids the vanishing gradient problem.
The **Softmax layer** predicts the person's identity.
This improves facial recognition accuracy.

WORK FLOW:



DATASET:

Dataset Name: `face_features.csv`

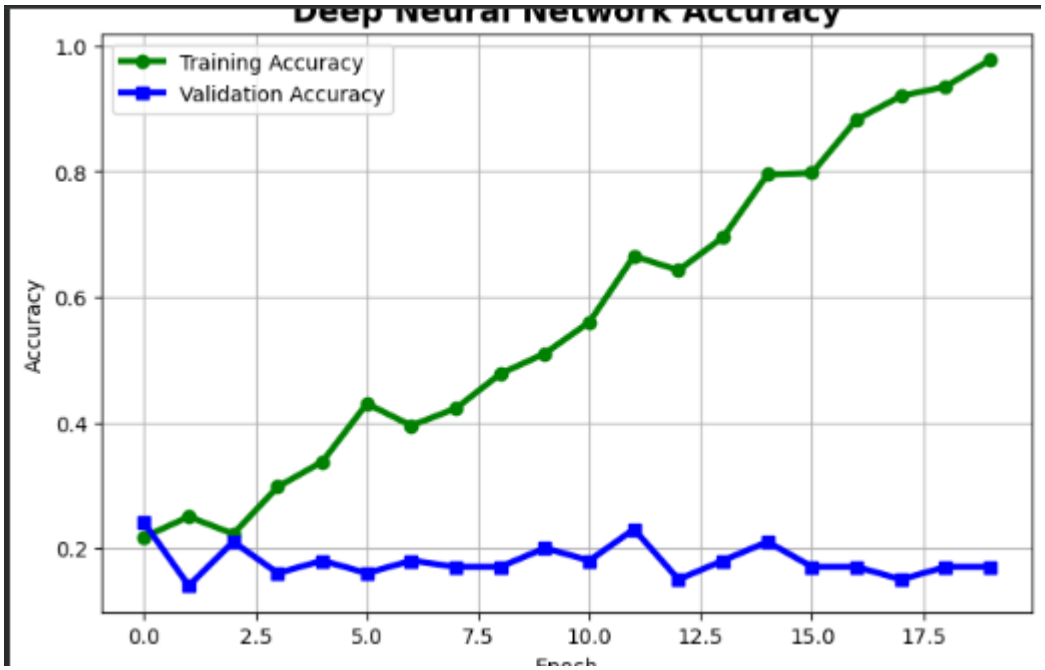
ImageID	EyeDistance	NoseWidth	FaceWidth	FaceHeight	LipWidth	Person
1	34	18	120	160	45	Person1
2	36	20	118	158	47	Person2
3	35	19	122	162	46	Person3
4	33	18	121	159	44	Person4
5	37	21	119	161	48	Person5
6	34	19	120	160	45	Person1
7	36	20	121	163	47	Person2
8	35	18	122	161	46	Person3
9	34	19	120	159	45	Person4
10	37	20	123	162	48	Person5

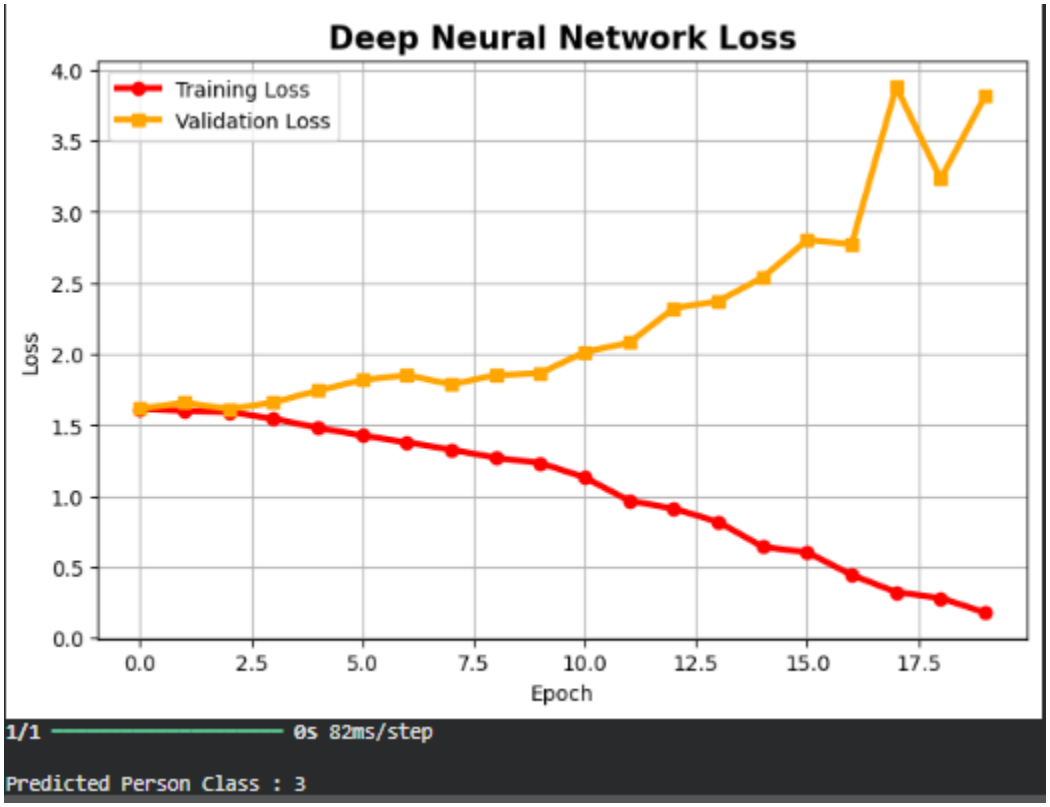
PYTHONCODE:

```
# Facial Recognition using Deep Neural Network

import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import to_categorical

np.random.seed(42)

X = np.random.rand(500, 100)

y = np.random.randint(0, 5, 500)
y = to_categorical(y)

print("Input Shape :", X.shape)
print("Output Shape :", y.shape)

model = Sequential()

model.add(Dense(256, activation='relu', input_shape=(100,)))
model.add(Dense(128, activation='relu'))
model.add(Dense(64, activation='relu'))
model.add(Dense(32, activation='relu'))
model.add(Dense(5, activation='softmax'))

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    X,
    y,
    epochs=20,
    batch_size=16,
    validation_split=0.2,
    verbose=1
)

plt.figure(figsize=(8,5))

plt.plot(
    history.history['accuracy'],
    color='green',
    marker='o',
    linewidth=3,
    label='Training Accuracy'
)
```

```
plt.plot(
    history.history['val_accuracy'],
    color='blue',
    marker='s',
    linewidth=3,
    label='Validation Accuracy'
)

plt.title("Deep Neural Network Accuracy", fontsize=15, fontweight='bold')
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.grid(True)
plt.legend()
plt.show()

plt.figure(figsize=(8,5))

plt.plot(
    history.history['loss'],
    color='red',
    marker='o',
    linewidth=3,
    label='Training Loss'
)

plt.plot(
    history.history['val_loss'],
    color='orange',
    marker='s',
    linewidth=3,
    label='Validation Loss'
)

plt.title("Deep Neural Network Loss", fontsize=15, fontweight='bold')
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid(True)
plt.legend()
plt.show()

sample = np.random.rand(1, 100)

prediction = model.predict(sample)

predicted_person = np.argmax(prediction)

print("\nPredicted Person Class :", predicted_person)
```

OUTPUT:


Deep Neural Network Accuracy
Training Accuracy
Validation Accuracy
Accuracy
Epoch


Deep Neural Network Loss
Training Loss
Validation Loss
Loss
Epoch
1/1 0s 82ms/step
Predicted Person Class : 3

4. Restricted Boltzmann Machines (RBMs), Unsupervised Training of Neural

Networks

A streaming service wants to recommend movies to users based on their viewing

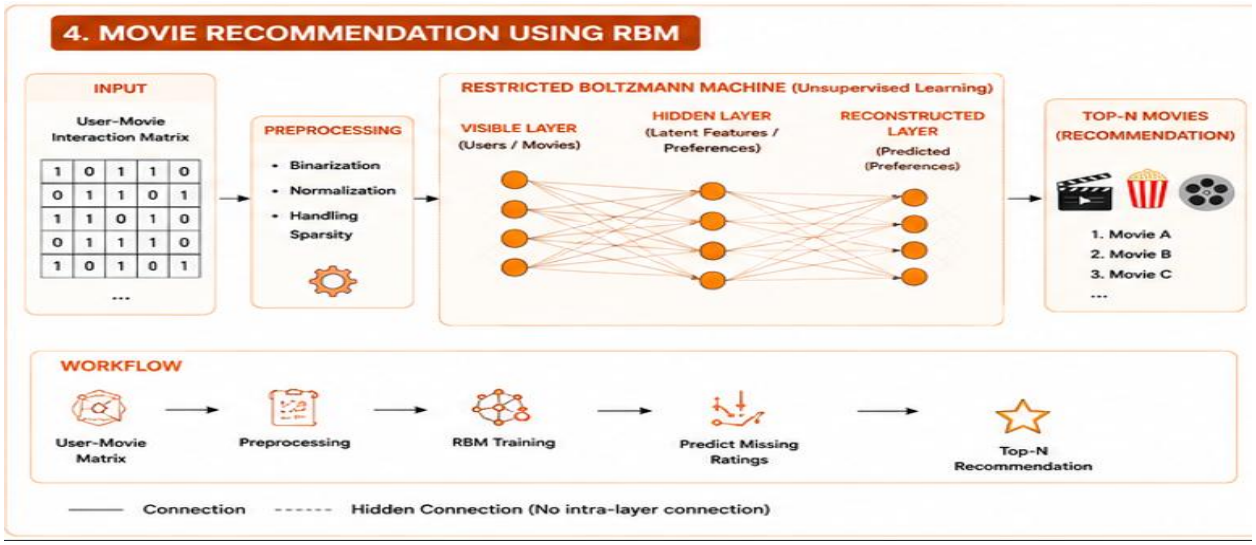
history without using labeled data. Explain how RBMs can be applied to build the

recommendation engine.

Explanation:

- Users' movie viewing history is stored in a user-movie matrix.
- An **RBM** learns hidden relationships between users and movies.
- It identifies user preferences without labeled data.
- The model predicts movies a user is likely to watch.
- Finally, the system recommends the top-rated movies.

WORKFLOW:



DATASET:

Dataset Name: `movie_ratings.csv`

User	MovieA	MovieB	MovieC	MovieD	MovieE
User1	1	1	0	1	0
User2	1	0	1	1	1
User3	0	1	1	0	1
User4	1	1	1	1	0
User5	0	0	1	1	1
User6	1	0	0	1	1
User7	0	1	1	1	0
User8	1	1	0	0	1
User9	1	0	1	1	0
User10	0	1	0	1	1

PYTHON CODE:

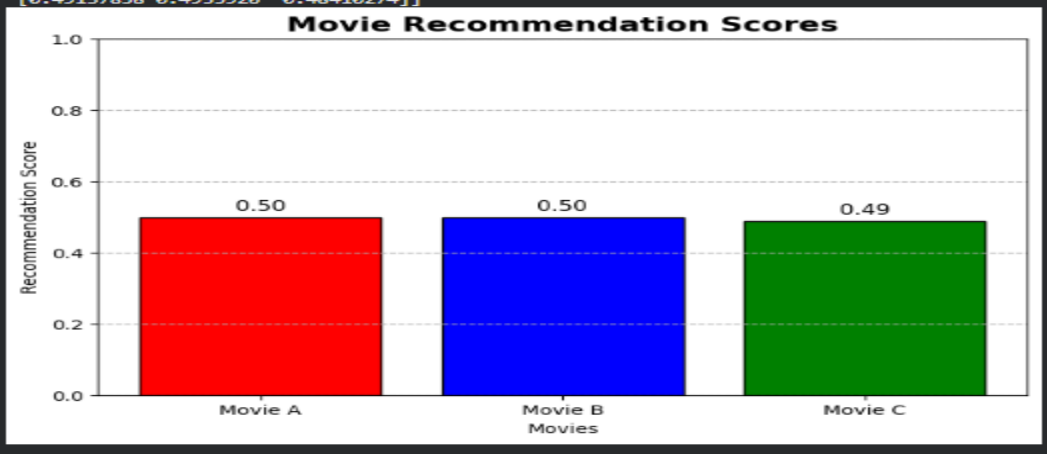
```
# Movie Recommendation using Restricted Boltzmann Machine (RBM)

import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import BernoulliRBM

# User-Movie Rating Matrix
X = np.array([
    [1,1,0,1,0],
    [1,0,1,1,1],
    [0,1,1,0,1],
    [1,1,1,1,0],
    [0,0,1,1,1],
    [1,0,0,1,1],
    [0,1,1,1,0],
    [1,1,0,0,1]
```

```
])

print("User-Movie Matrix")
print(X)

# Create RBM Model
rbm = BernoulliRBM(
    n_components=3,
    learning_rate=0.01,
    n_iter=30,
    random_state=42
)

# Train RBM
rbm.fit(X)

# Hidden Features
hidden = rbm.transform(X)

print("\nHidden Feature Matrix")
print(hidden)

# Average Recommendation Score
scores = np.mean(hidden, axis=0)

movies = ["Movie A", "Movie B", "Movie C"]

# Plot Recommendation Scores
plt.figure(figsize=(8,5))

bars = plt.bar(
    movies,
    scores,
    color=["red","blue","green"],
    edgecolor="black"
)

plt.title("Movie Recommendation Scores",
          fontsize=15,
          fontweight="bold")

plt.xlabel("Movies")
plt.ylabel("Recommendation Score")

plt.ylim(0,1)
plt.grid(axis='y', linestyle='--')

# Display score on bars
for bar in bars:
    y = bar.get_height()
    plt.text(
        bar.get_x()+bar.get_width()/2,
        y+0.02,
        f"{y:.2f}",
```

```
        ha='center',
        fontsize=11
    )

plt.show()

# Recommend Best Movie
best_movie = movies[np.argmax(scores)]

print("\nRecommended Movie :", best_movie)
```

OUTPUT:

```
User-Movie Matrix
[[1 1 0 1 0]
 [1 0 1 1 1]
 [0 1 1 0 1]
 [1 1 1 1 0]
 [0 0 1 1 1]
 [1 0 0 1 1]
 [0 1 1 1 0]
 [1 1 0 0 1]]

Hidden Feature Matrix
[[0.49954366 0.49684408 0.48731592]
 [0.50349336 0.49869134 0.4871546 ]
 [0.49222906 0.49858251 0.48634242]
 [0.50170984 0.49927354 0.48832589]
 [0.50217793 0.49925186 0.48838495]
 [0.50132723 0.49626191 0.48614476]
 [0.50039437 0.49983406 0.48955636]
 [0.49137858 0.4955926  0.48410274]]
```




```
Recommended Movie : Movie A
```

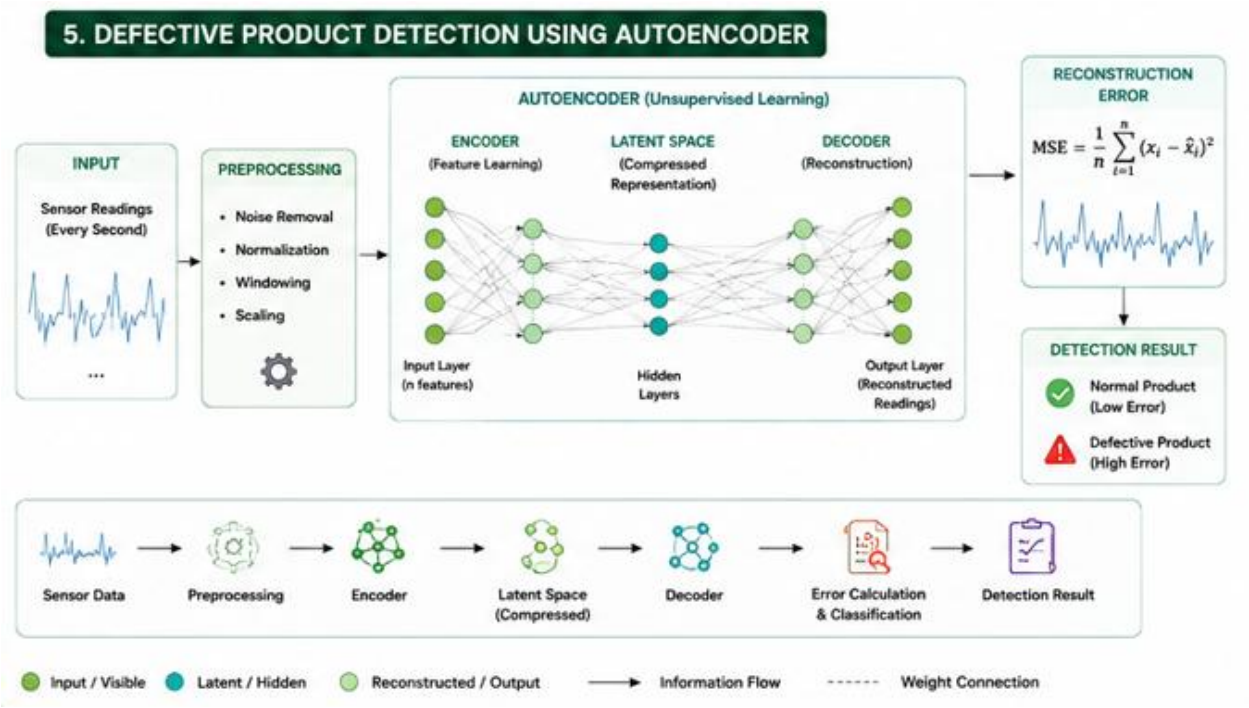
5.Deep Learning Applications, Representation Learning
A manufacturing company wants to detect defective products from sensor readings
collected every second. Design a deep learning solution using concepts from
representation learning and neural networks.

Explanation:

- Sensor readings are collected continuously from manufacturing machines.

- An **Autoencoder** learns the pattern of normal sensor data.
- It reconstructs the sensor readings after training.
- If the reconstruction error is small, the product is normal.
- If the error is large, the product is classified as defective.

WORKFLOW:



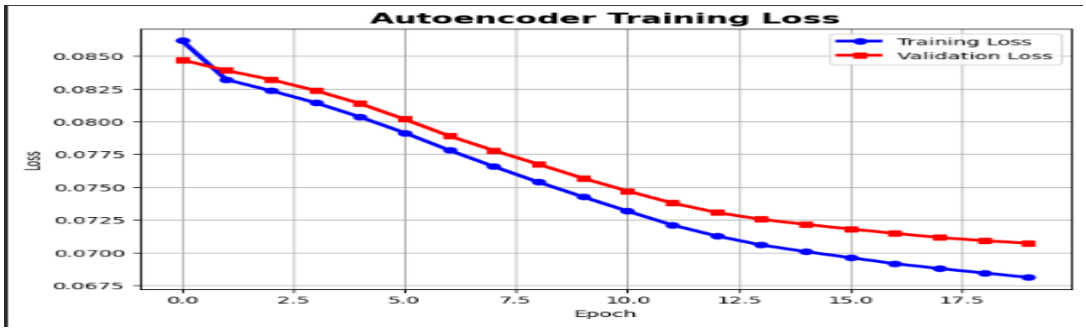
PYTHON CODE:

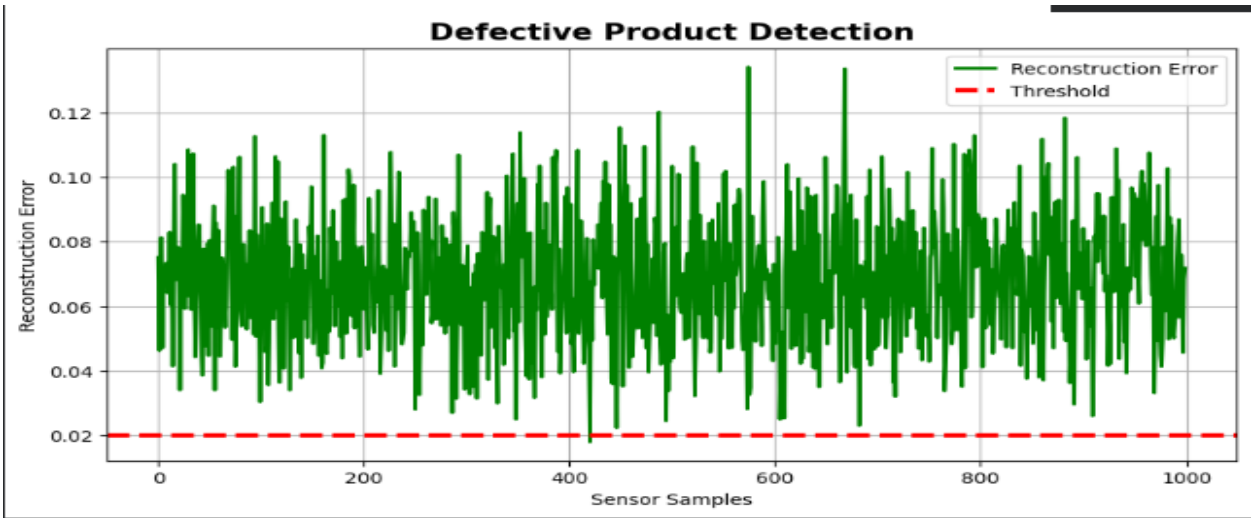
```
# Defective Product Detection using Autoencoder

import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Generate Sample Sensor Data
np.random.seed(42)
X = np.random.rand(1000, 15)

print("Sensor Data Shape :", X.shape)

# Build Autoencoder
model = Sequential()

model.add(Dense(8, activation='relu', input_shape=(15,)))
model.add(Dense(4, activation='relu'))
```

```
model.add(Dense(8, activation='relu'))
model.add(Dense(15, activation='sigmoid'))

# Compile Model
model.compile(optimizer='adam', loss='mse')

# Train Model
history = model.fit(
    X,
    X,
    epochs=20,
    batch_size=16,
    validation_split=0.2,
    verbose=1
)

# Reconstruct Sensor Data
predicted = model.predict(X)

# Calculate Reconstruction Error
error = np.mean((X - predicted) ** 2, axis=1)

threshold = 0.02

# Plot Training Loss
plt.figure(figsize=(8,5))

plt.plot(
    history.history['loss'],
    color='blue',
    marker='o',
    linewidth=3,
    label='Training Loss'
)

plt.plot(
    history.history['val_loss'],
    color='red',
    marker='s',
    linewidth=3,
    label='Validation Loss'
)

plt.title("Autoencoder Training Loss", fontsize=15, fontweight='bold')
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid(True)
plt.legend()
plt.show()

# Plot Reconstruction Error
plt.figure(figsize=(10,5))

plt.plot(
```

```
    error,
    color='green',
    linewidth=2,
    label='Reconstruction Error'
)

plt.axhline(
    y=threshold,
    color='red',
    linestyle='--',
    linewidth=3,
    label='Threshold'
)

plt.title("Defective Product Detection", fontsize=15, fontweight='bold')
plt.xlabel("Sensor Samples")
plt.ylabel("Reconstruction Error")
plt.grid(True)
plt.legend()
plt.show()

# Detect Products
print("\nProduct Status\n")

for i in range(10):
    if error[i] > threshold:
        print("Product", i+1, ": Defective")
    else:
        print("Product", i+1, ": Normal")

print("\nDetection Completed Successfully.")
```

OUTPUT:






```
Product Status

Product 1 : Defective
Product 2 : Defective
Product 3 : Defective
Product 4 : Defective
Product 5 : Defective
Product 6 : Defective
Product 7 : Defective
Product 8 : Defective
Product 9 : Defective
Product 10 : Defective

Detection Completed Successfully.
```